1 INTRODUÇÃO

Este documento apresenta o item B2 (camada de acesso a dados) da Etapa 2 do projeto final da disciplina de Laboratório de Banco de Dados, conforme o enunciado (MOURA JÚNIOR, 2026). O sistema desenvolvido é o Sistema de Controle de Vendas Toy Storey, uma aplicação em Java conectada a um banco MySQL 8.

Para este item, o enunciado exige o uso de consultas parametrizadas (prepared statements), a demonstração de uma tentativa de injeção de SQL que a aplicação resiste e o armazenamento das credenciais do banco fora do código-fonte. As seções seguintes descrevem como cada exigência foi atendida.

2 ORGANIZAÇÃO DA CAMADA DE ACESSO

Todo o código que acessa o MySQL está no pacote br.com.entidade, escrito pela equipe em JDBC puro. A classe DAO é a base: lê a configuração, abre e fecha a conexão. Cada entidade tem uma classe CRUD* que herda de DAO e concentra os comandos SQL daquela entidade. As telas, no pacote br.com.visao, não montam SQL e chamam apenas os métodos dessas classes.

A conexão é aberta com o usuário MySQL correspondente ao perfil de quem fez login (Sessao.getPerfilBanco()), e não com um usuário único da aplicação. Isso aplica o princípio do menor privilégio já na camada de acesso, conforme o Quadro 1.

Quadro 1 – Perfis de acesso ao banco

Perfil	Usuário MySQL	Acesso concedido
Gerente	app_gerente	Leitura e escrita no esquema; exclusão apenas nas tabelas de cadastro (venda não se apaga, se cancela)
Operador	app_operador	Colunas e tabelas necessárias ao registro de vendas e a visão de atendimento de clientes
Leitura	app_leitura	Somente a visão vw_resumo_vendas_diarias
Autenticação	app_auth	Somente a visão vw_login

Fonte: elaborado pelos autores (2026).

Ao abrir a conexão, o DAO também grava o funcionário da sessão na variável @id_funcionario_app, por meio de comando parametrizado, para que os gatilhos de auditoria identifiquem quem fez cada alteração.

3 USO DE CONSULTAS PARAMETRIZADAS

Todas as instruções SQL da aplicação são executadas com PreparedStatement. Os valores informados pelo usuário entram sempre por setString, setInt, setDate, setTimestamp e métodos equivalentes, nunca dentro do texto do SQL. Uma busca no código-fonte não encontra nenhuma chamada a createStatement, e nenhum executeQuery ou executeUpdate recebe texto SQL como argumento. O Quadro 2 resume as classes da camada.

Quadro 2 – Classes de acesso a dados e técnica empregada

Classe	Operações	Observação
CRUDCategoria	Inserir, alterar, excluir e buscar	Parâmetros posicionais (?)
CRUDProduto	CRUD completo, busca por identificador e por código de barras, listagem	Parâmetros posicionais (?)
CRUDCliente e CRUDFuncionario	CRUD sobre pessoa e sobre a tabela da especialização	Duas tabelas na mesma transação
CRUDVenda	Venda que altera várias tabelas	setAutoCommit(false), COMMIT, ROLLBACK e FOR UPDATE no estoque
CRUDConsulta	Duas consultas com junção e agregação e o relatório sobre a visão	Filtros opcionais, ver subseção 3.1
CRUDLogin	Autenticação	WHERE login = ?, lendo apenas vw_login

Fonte: elaborado pelos autores (2026).

3.1 Consultas com filtro opcional

Em CRUDConsulta, o texto do SQL cresce com fragmentos fixos escritos no código, como AND pg.forma_pagamento = ?, e o valor do filtro é ligado depois com setString ou setInt. O operador + aparece no código apenas para juntar literais fixos, por exemplo SELECT_BASE + "WHERE c.cpf = ?", e nunca para inserir dados vindos da tela.

4 CREDENCIAIS FORA DO CÓDIGO-FONTE

O DAO procura cada credencial primeiro em variável de ambiente (DB_URL, DB_<PERFIL>_USER e DB_<PERFIL>_PASSWORD) e, se ela não existir, no arquivo

config.properties do diretório de execução. Nenhuma URL, usuário ou senha aparece no código Java.

O repositório versiona somente o arquivo config.example.properties, que contém valores de exemplo. O arquivo real, config.properties, está listado no .gitignore e permanece apenas na máquina de quem executa a aplicação. O arquivo README.md descreve o passo de copiar o arquivo de exemplo e preencher as senhas.

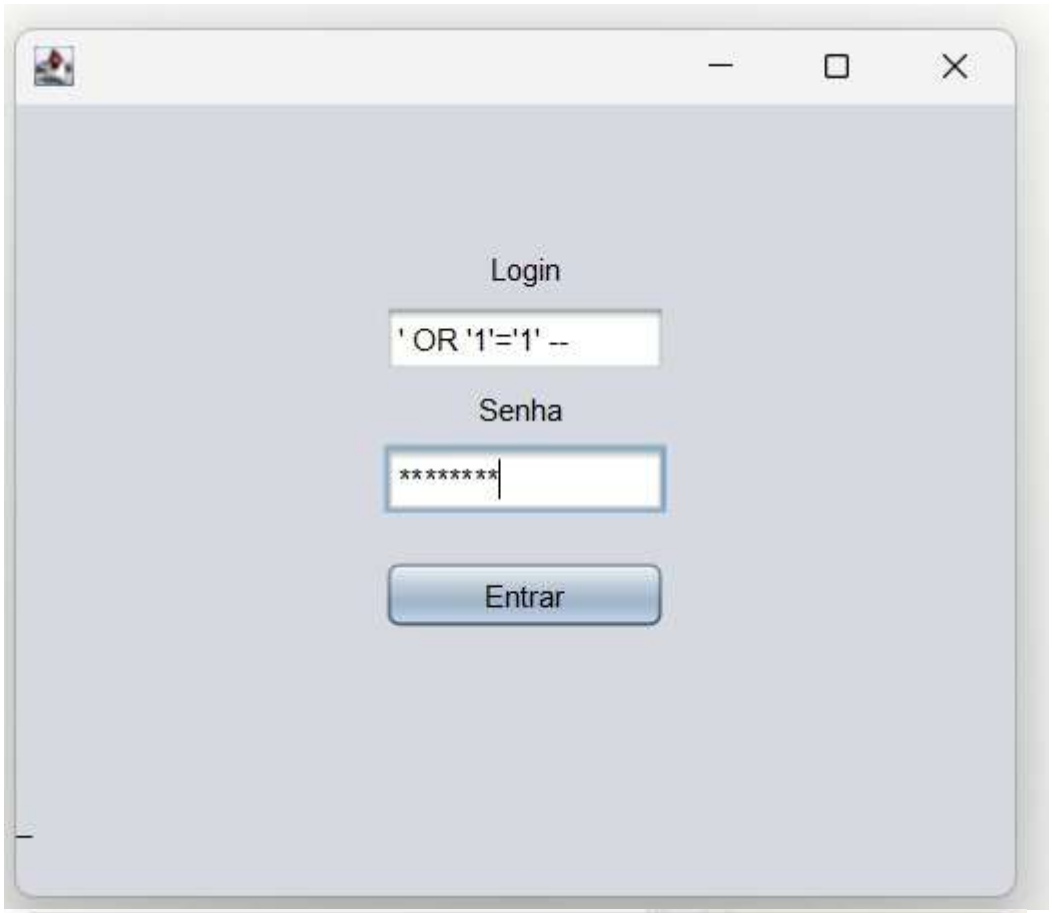
5 DEMONSTRAÇÃO DE TENTATIVA DE INJEÇÃO DE SQL

Os testes foram feitos com a aplicação em execução e o banco loja_toystorey povoado com o script 02_carga.sql. O Quadro 3 e o Quadro 4 registram a entrada usada, o comando enviado ao banco e o resultado de cada tentativa.

5.1 Teste 1 – campo de login

Quadro 3 – Tentativa de injeção de SQL no campo de login

Item	Conteúdo
Tela	Login
Login digitado	' OR '1'='1' --
Senha digitada	qualquer
SQL enviado ao banco	SELECT id_pessoa, nome, senha_hash, cargo FROM vw_login WHERE login = ?
Valor do parâmetro	O texto digitado, inteiro, tratado como dado
Resultado esperado	Nenhuma linha encontrada; a tela exibe “Login ou senha inválidos.”

Item	Conteúdo
Resultado obtido	 

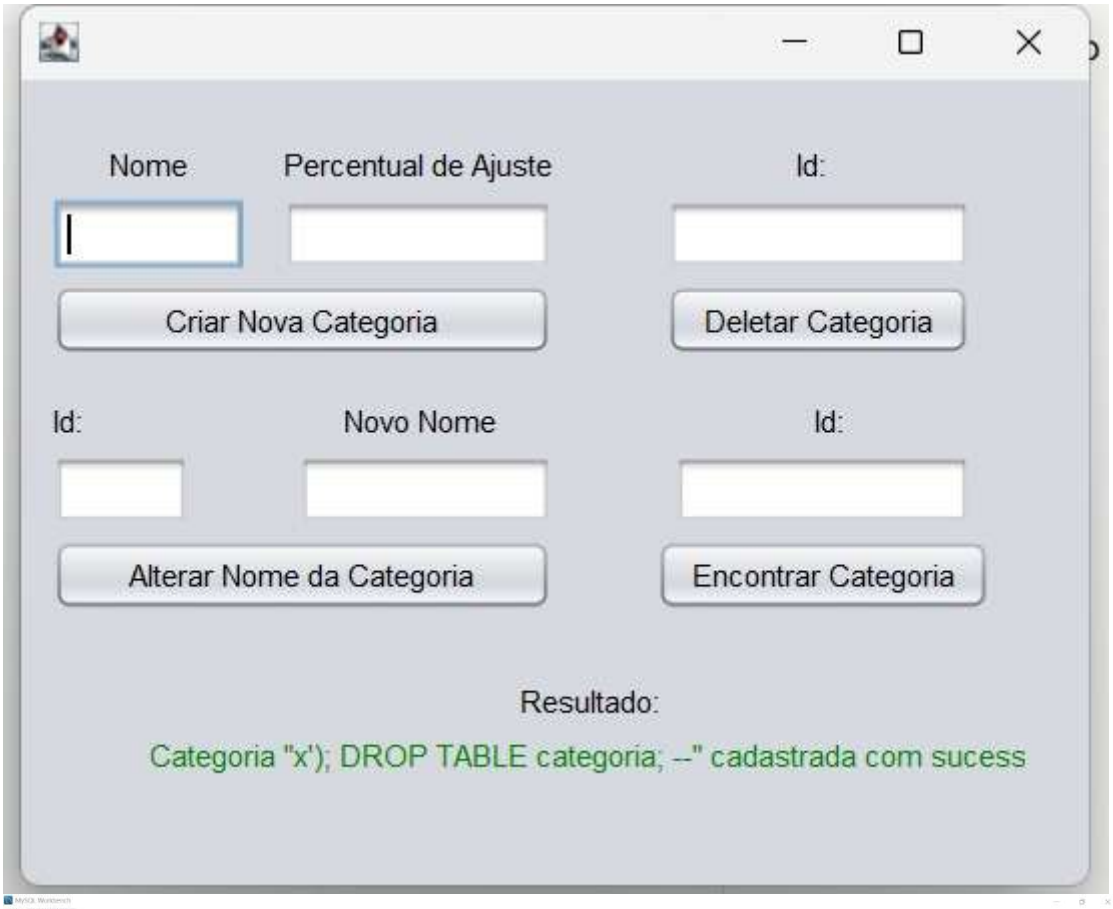
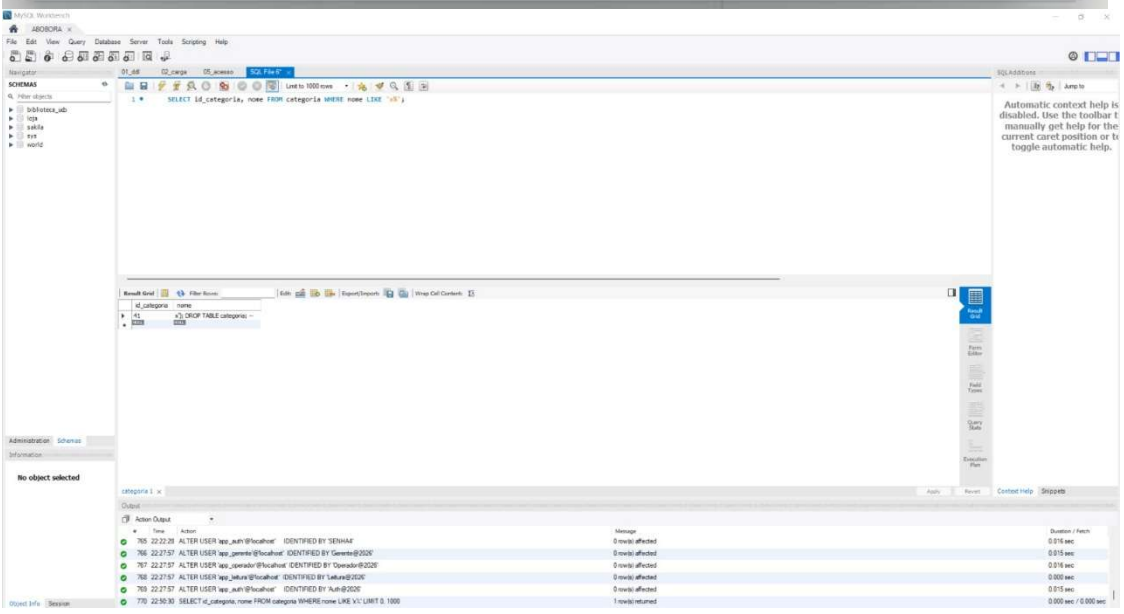
Fonte: elaborado pelos autores (2026).

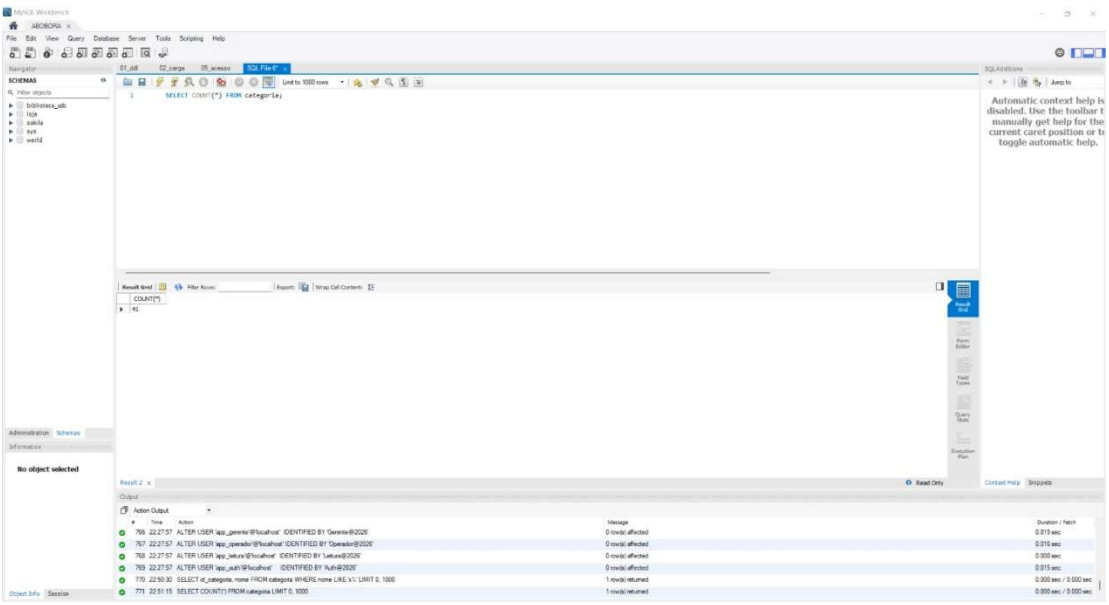
Se o SQL fosse montado por concatenação, o banco receberia WHERE login = '' OR '1'='1' -- ', condição verdadeira para todos os funcionários. Com o parâmetro, a comparação é feita contra a cadeia de caracteres completa, que não corresponde a nenhum login.

5.2 Teste 2 – texto gravado no banco

Quadro 4 – Tentativa de injeção de SQL no cadastro de categoria

Item	Conteúdo
Tela	Cadastro de categoria
Nome digitado	x'); DROP TABLE categoria; --
SQL enviado ao banco	INSERT INTO categoria(nome, percentual_reajuste_padrao) values(?, ?)
Resultado esperado	A categoria é gravada com esse texto literal e a tabela categoria continua existindo
Verificação	SELECT id_categoria, nome FROM categoria WHERE nome LIKE 'x%'; e SELECT COUNT(*) FROM categoria;

Resultado obtido	 

Item	Conteúdo
	 

Fonte: elaborado pelos autores (2026).

Após a demonstração, o registro de teste foi excluído pela tela de categorias.

5.3 Camadas adicionais de proteção

Além da parametrização, o projeto conta com outras barreiras, listadas a seguir:

a) o usuário app_auth, usado no login, só consegue ler a visão vw_login;
b) nenhum dos usuários da aplicação possui o privilégio DROP;
c) a URL de conexão não habilita allowMultiQueries, portanto o driver não aceita várias instruções em uma única chamada;
d) o CPF é validado por expressão regular (11 dígitos) antes de chegar ao banco, o que complementa a parametrização, mas não a substitui.